SAGAR INSTITUTE OF SCIENCE & TECHNOLOGY (SISTec)

DEPARTMENT OF CSE - CYBER SECURITY

QUESTION BANK SOLUTIONS — SESSION 2024-25

COMPUTER ORGANIZATION & ARCHITECTURE (CY-603)

Document Curated & Published by: EMo Learners

Subject: Computer Organization & Architecture (CS-404)

Total Solved Questions: 20

Design Standards: Highly Spacious Step-by-Step Traces

Branding Highlight: EMo Learners Premium Quality

UNIT 1: COMPUTER STRUCTURE, BUSES, AND CONTROL UNITS

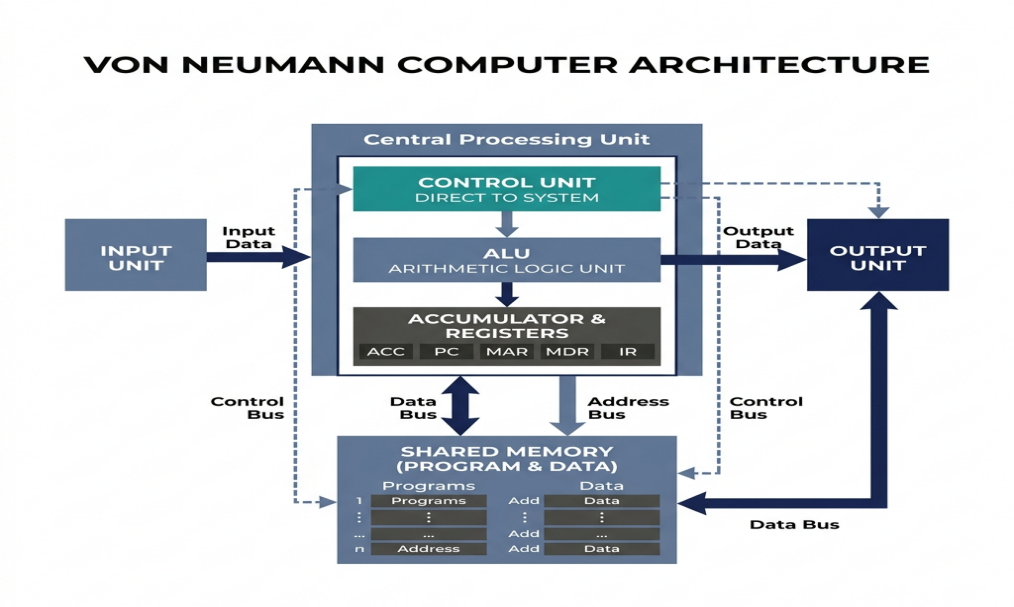
Q.1 **Explain the structure of a desktop computer with a neat block diagram and describe the function of each major unit.**

Bloom's Taxonomy: 1(Remembering) | Course Outcome: CO1

Answer:

A desktop computer is structured around a central motherboard housing the processing core, memory systems, and interface cards.

Functional Block Diagram (Von-Neumann Architecture):



Functions of Major Units:

• Input Unit: Translates user data (keyboard, mouse) into binary representation for processing.

- Memory Unit: Stores operating systems, active applications (RAM), and booting code (ROM).
- CPU Control Unit: Decodes instructions and generates timing signals to coordinate data flows.
- CPU ALU Unit: Execution core performing mathematical and logical processing on operands.
- Output Unit: Converts binary calculations back into human-readable text/displays (monitors, printers).

Q.2 Define Program Counter, Instruction Register, Memory Register, Control Word, and Stack Organization with suitable examples.

Bloom's Taxonomy: 1(Remembering) | Course Outcome: CO1

Answer:

Essential registers and memory structures are defined below:

- Program Counter (PC): A 12-bit or 16-bit register holding the memory address of the next instruction to fetch. Example: If PC = `0AF2H`, the CPU fetches the instruction word at memory address `0AF2H`.
- Instruction Register (IR): Holds the active instruction code fetched from memory during decoding. Example: Stores the 16-bit binary instruction `7020H` (representing clear accumulator).
- Memory Register (MBR / Data Register): Temporarily stores data retrieved from or written to memory address slots. Example: Stores the 16-bit word loaded from RAM address `1F0H` before processing.
- Control Word: A sequence of control bits stored in control memory that directly drives hardware multiplexers and enable lines. Example: A 16-bit control word where bits 0–2 select ALU function, bits 3–5 select source register, and bit 6 enables register load.
- Stack Organization: A LIFO (Last-In, First-Out) memory stack managed by a **Stack Pointer (SP)** register. Example: In subroutine calls, the return address is 'Pushed' onto the stack (decrementing SP) and later 'Popped' (incrementing SP) to resume main program execution.

Q.3 Explain the general register organization of CPU and describe the role of ALU in instruction execution.

Bloom's Taxonomy: 2(Understanding) | Course Outcome: CO1

Answer:

In a **General Register Organization**, registers share access to an internal bus structure that feeds directly into the ALU.

Bus-based CPU Register Organization:

Outputs from registers are routed to two multiplexers (MUX A and MUX B). The select inputs (SELA and SELB) choose two registers to place their data on Bus A and Bus B. The ALU processes these buses based on selection variables (OPR) and writes the output sum back into a target register via a decoder (SELD) on the active clock edge.

- Role of ALU in Instruction Execution: The ALU acts as the central execution engine. Once the control unit decodes an instruction (e.g. `ADD R1, R2`), it places the contents of R1 on Bus A, R2 on Bus B, selects the addition operation in the ALU, and routes the sum back to R1. This executes the target machine instruction in a single

clock cycle.

Q.4 **Describe the instruction format and various addressing modes used in computer systems with examples.**

Bloom's Taxonomy: 2(Understanding) | Course Outcome: CO1

Answer:

An **Instruction Format** defines the bit division of a computer instruction word. A typical format consists of an **Opcode** (the operation), an **Addressing Mode** (specifying address calculation rules), and **Operands** (data registers or addresses).

Core Addressing Modes:

- Implied Mode: The operand is implicitly defined in the opcode itself. Example: `CMA` (Complement Accumulator) requires no address field.
- Immediate Mode: The operand is explicitly specified in the instruction itself. Example: `ADD 5` adds the value 5 directly to the accumulator.
- Direct Addressing: The address field contains the actual address of the operand. Example: `ADD 250H` fetches the value at memory address `250H`.
- Indirect Addressing: The address field points to a memory location that contains the actual effective address of the operand. Example: `ADD [250H]` looks at address `250H` to read address `500H`, then fetches the operand from `500H`.
- Register Indirect: A register holds the address of the operand in memory. Example: `ADD (R1)` fetches the operand at memory address stored inside R1.

Q.5 **Illustrate the concept of Register Transfer Language (RTL) and demonstrate bus and memory transfer operations using suitable examples.**

Bloom's Taxonomy: 3(Applying) | Course Outcome: CO1

Answer:

Register Transfer Language (RTL) is a formal symbolic notation used to describe the transfer of binary information between registers in a digital system.

Demonstration of Transfer Operations:

- Register Transfer: `R2 ← R1`
  Moves the 16-bit binary contents of R1 directly into R2 during the next active clock transition.
- Bus Transfer (Using Common Bus): `BUS ← R1, R2 ← BUS`
  Register R1 places its output on the common bus lines, and register R2 loads the data from the bus.
- Memory Read: `DR ← M[AR]`
  The Data Register (DR) loads the data word from the memory address specified by the Address Register (AR).

• Memory Write: `M[AR] ← R1`
Writes the contents of register R1 into the memory address specified by the AR.

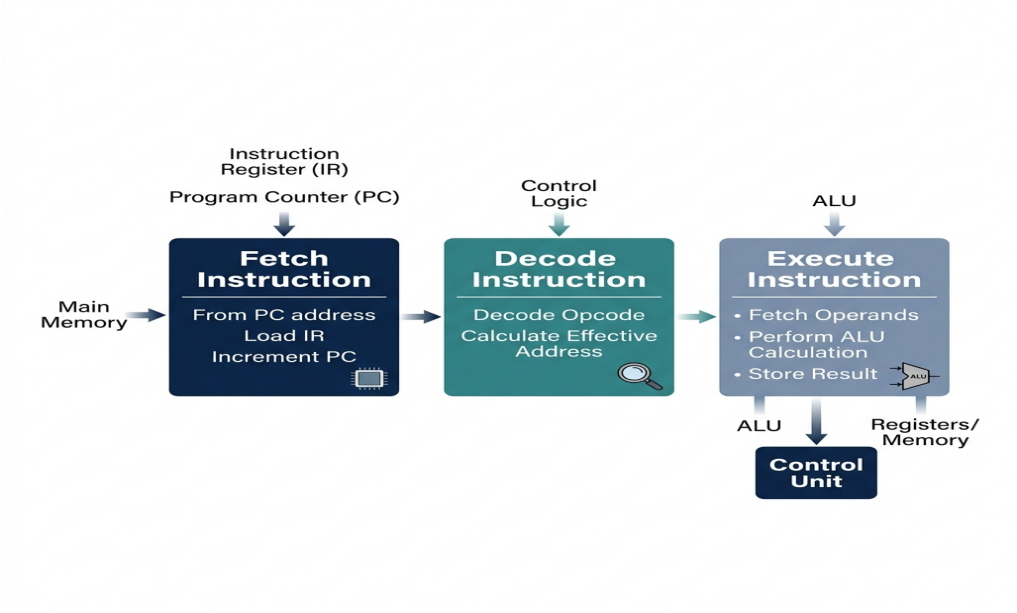
Q.6 **Demonstrate the fetch and execution cycle of an instruction with the help of a flow diagram.**

Bloom's Taxonomy: 3(Applying) | Course Outcome: CO1

Answer:

The **Fetch-Execute Cycle** represents the continuous operational loop of the CPU.

Instruction Cycle Flowchart:



Q.7 **Analyze the bus structure of a computer system and examine how CPU and memory communicate through system buses.**

Bloom's Taxonomy: 4(Analyzing) | Course Outcome: CO1

Answer:

A computer system uses three primary buses (system bus) to manage communication between the CPU and memory:

• Address Bus: A unidirectional bus carrying the target memory address from the CPU to memory chip decoders.

• Data Bus: A bidirectional bus transferring the actual instruction or operand data word between the CPU and memory.

• Control Bus: Unidirectional control lines carrying timing and operational commands (such as **Memory Read (MEMR)** or **Memory Write (MEMW)**) from the CPU control unit to synchronize the memory interface.

Step-by-Step Read Communication Sequence:

1. The CPU writes the target address onto the address bus.
2. The CPU activates the **Memory Read** line on the control bus.
3. The memory decoder decodes the address bus inputs and enables the target memory location.
4. The memory chip drives the data contents onto the data bus.
5. The CPU reads the bits off the data bus and loads them into internal registers (like the MBR).

Q.8 **Compare hardwired control unit and microprogrammed control unit with respect to design complexity, speed, and flexibility.**

Bloom's Taxonomy: 4(Analyzing) | Course Outcome: CO1

Answer:

Hardwired and microprogrammed control units represent the two primary design styles for processor control units. A spacious comparison is tabulated below:

Comparison Parameter	Hardwired Control Unit	Microprogrammed Control Unit
Design Complexity	Highly complex. Sprawling logic gates are difficult to design and debug.	Systematic, simple, and clean design utilizing standard memory arrays.
Speed	Extremely Fast: No memory access latency; signals propagate through gates instantly.	Slower: Must access internal Control Memory (ROM) for every microinstruction step.
Flexibility	Low: Adding new instructions requires physical redesign and rewiring.	High: Easily update or add instructions by modifying microprograms in ROM.
Typical Architecture	Used in modern high-speed **RISC** processors.	Ideal for complex **CISC** processors.

Q.9 **Evaluate the role of microprogram sequencer and control memory in sequencing and execution of microinstructions.**

Bloom's Taxonomy: 5(Evaluating) | Course Outcome: CO1

Answer:

In a microprogrammed control unit, execution is guided by the **Control Memory** and the **Microprogram Sequencer**:

- Control Memory (ROM): An internal ROM array holding the sequence of control words (microinstructions) that execute every machine opcode.
- Microprogram Sequencer: An address-selection unit that determines the next address to load into the Control Address Register (CAR). It evaluates condition bits, control codes, and branch flags to select the next address

from four possible sources:

1. **CAR + 1:** Increments CAR to execute the next sequential microinstruction.
2. **Address Field of Microinstruction:** Performs a jump to a branch address specified in the current control word.
3. **Opcode Mapping Logic:** Converts the opcode of the machine instruction inside the IR into a starting address in Control Memory.
4. **Subroutine Return (SBR):** Loads the return address from the Subroutine Register to return from a microprogram function.

Q.10 Design a functional architecture of a control unit incorporating microinstruction format and control memory organization.

Bloom's Taxonomy: 6(Creating) | Course Outcome: CO1

Answer:

Designing a microprogrammed control unit requires defining a **Control Memory structure** and a **Microinstruction format** to coordinate data paths.

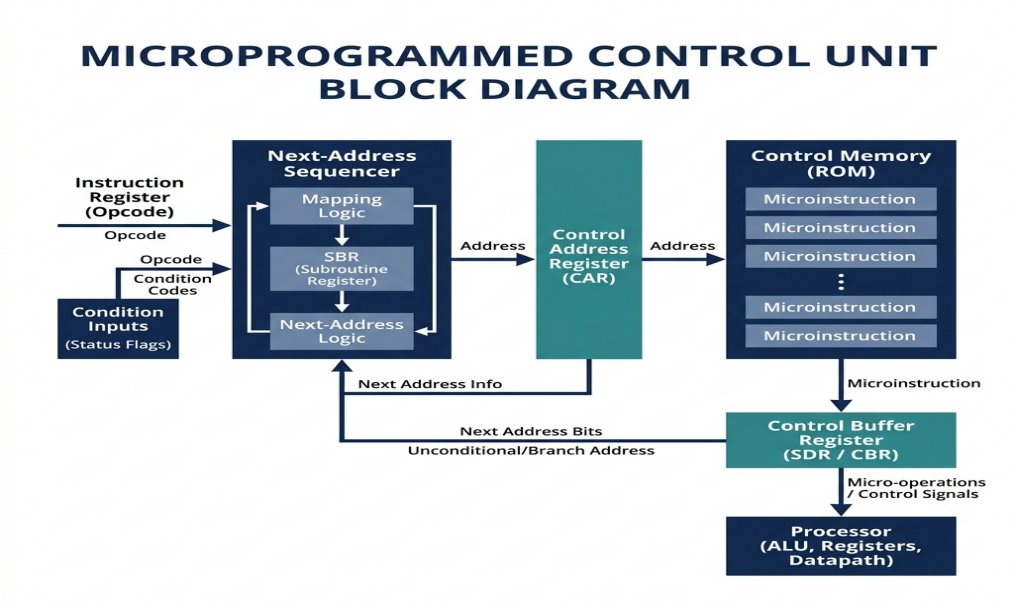
1. Microinstruction Word Format Design (20 bits):

• Microoperation Fields (F1, F2): Decode to enable registers or ALU functions.

• Condition Field (CD): 00 = Always, 01 = If Zero, 10 = If Carry, 11 = If Sign.

• Address Field (AD): Holds a 6-bit target address in Control Memory (up to 64 microinstructions).

2. Functional Architecture Diagram:



UNIT 2: COMPUTER ARITHMETIC & NUMERICAL OPERATIONS

Q.1 **Define 1’s complement and 2’s complement representation and list their advantages in signed arithmetic operations.**

Bloom's Taxonomy: 1(Remembering) | Course Outcome: CO2

Answer:

1's and 2's complement representations are used to represent signed integers in binary computer systems.

- 1's Complement: Formed by inverting all the bits of a positive binary number (0s become 1s, 1s become 0s).
- 2's Complement: Formed by taking the 1's complement and adding 1 to the Least Significant Bit (LSB).

Advantages of 2's Complement in Signed Arithmetic:

- Unique Representation of Zero: 1's complement has two zero representations (+0: `00000000` and -0: `11111111`), which requires complicated comparisons. 2's complement has a single representation for zero (`00000000`), simplifying logic.
- Unified Addition & Subtraction Hardware: Subtraction A - B is calculated directly by adding A + $(-B)_{2's}$. The CPU does not require separate adder and subtractor logic circuits; a single parallel binary adder is used for both operations.
- No End-Around Carry: During addition, any carry-out from the sign bit (MSB) is simply discarded, unlike 1's complement which requires adding the carry back to the sum (end-around carry).

Q.2 **State the algorithmic steps involved in Booth's multiplication algorithm.**

Bloom's Taxonomy: 1(Remembering) | Course Outcome: CO2

Answer:

Booth's Multiplication Algorithm multiplies two signed binary numbers in 2's complement format. The step-by-step algorithmic procedure is outlined below:

1. **Setup Registers:** Accumulator (A) = 0, Q_{-1} register = 0, Multiplier loaded into register Q, Multiplicand loaded into register M, and Sequence Counter (SC) set to bit length n.
2. **Loop Execution:** While SC > 0, examine the least significant bit of Q (Q_0) and Q_{-1}:

- **01:** Add multiplicand to A (`A = A + M`).
- **10:** Subtract multiplicand from A (`A = A - M` or `A = A + M' + 1`).
- **00 or 11:** Perform no arithmetic operation.

3. **Shift Phase:** Perform an **Arithmetic Right Shift (ARS)** on the combined registers A-Q-Q_{-1} by 1 bit, preserving the sign bit (MSB) of A.
4. **Decrement Counter:** Decrement SC by 1.
5. **Termination:** Once SC = 0, the final product is stored in the combined register space A Q.

Q.3 **Explain signed addition and subtraction using 2's complement method with suitable examples.**

Bloom's Taxonomy: 2(Understanding) | Course Outcome: CO2

Answer:

In 2's complement representation, subtraction is treated directly as addition, allowing unified hardware processing.

Demonstration with Examples:

Let us perform operations using 5-bit signed binary numbers (range: -16 to +15):

Example 1: Signed Addition (+9) + (+4)

- +9 in binary = 01001_2
  - +4 in binary = 00100_2
  - Perform direct binary addition:

```
  01001  (+9)
+ 00100  (+4)
-------
  01101  (+13 in decimal, MSB=0, positive). Correct!
```

Example 2: Signed Subtraction (+9) − (+4)

This is treated as **+9 + (-4)**:

- +9 in binary = 01001_2
- +4 in binary = 00100_2. Take 2's complement to represent -4:
  1's complement = 11011 → Add 1 = 11100_2.

• Perform binary addition:
  01001 (+9)
+ 11100 (-4)
---
(1)00101 (+5 in decimal, MSB=0, discard carry). Correct!

Q.4 **Describe the procedure of floating-point arithmetic operations in computer systems.**

Bloom's Taxonomy: 2(Understanding) | Course Outcome: CO2

Answer:

Floating-point numbers are represented in scientific format (sign, exponent, mantissa). Arithmetic operations process exponents and mantissas separately:

• Addition and Subtraction:

1. **Compare Exponents:** Compute exponent difference d = E1 - E2.
2. **Align Mantissas:** Shift the mantissa of the number with the smaller exponent right by d bits and set its exponent to the larger value.
3. **Add/Subtract:** Add or subtract the aligned mantissas based on their sign bits.
4. **Normalize:** If carry is generated, shift the mantissa right by 1 bit and increment the exponent. If there are leading zeros, shift left and decrement the exponent.

• Multiplication:

1. **Add Exponents:** Add exponents and subtract bias (bias = 127 in single-precision) to avoid double bias.
2. **Multiply Mantissas:** Multiply the fractional mantissas directly.
3. **Normalize & Round:** Normalize the product and round the mantissa to 23 bits.

• Division:

1. **Subtract Exponents:** Subtract divisor exponent from dividend exponent and add bias.
2. **Divide Mantissas:** Divide the dividend mantissa by the divisor mantissa.
3. **Normalize & Round:** Normalize and round the quotient.

Q.5 **Apply Booth's Algorithm to multiply two signed binary numbers.**

Bloom's Taxonomy: 3(Applying) | Course Outcome: CO2

Answer:

Let us multiply **(+13)** and **(-9)** using Booth's Algorithm in 6-bit signed 2's complement representation:

• Multiplicand (M) = +13 = 001101_2
• Negative Multiplicand (-M) = -13 = 110011_2 (2's complement of 001101)

- Multiplier (Q) = -9 = 110111_2 (2's complement of 001001)

• **Registers:** Accumulator (A) = 000000 (6 bits), Q_{-1} = 0 (1 bit), SC = 6.

Cycle-by-Cycle Tracing:

• Initial State: A = 000000, Q = 110111, Q_{-1} = 0, SC = 6

• Cycle 1:
- Check Q_0Q_{-1} = 10 → Perform A = A + (-M):
A = 000000 + 110011 = 110011.
- Perform **Arithmetic Right Shift (ARS)**:
A = 111001, Q = 111011, Q_{-1} = 1. SC = 5.

• Cycle 2:
- Check Q_0Q_{-1} = 11 → Perform **ARS only**:
A = 111100, Q = 111101, Q_{-1} = 1. SC = 4.

• Cycle 3:
- Check Q_0Q_{-1} = 11 → Perform **ARS only**:
A = 111110, Q = 011110, Q_{-1} = 1. SC = 3.

• Cycle 4:
- Check Q_0Q_{-1} = 01 → Perform A = A + M:
A = 111110 + 001101 = 001011 (discard carry).
- Perform **ARS**:
A = 000101, Q = 101111, Q_{-1} = 0. SC = 2.

• Cycle 5:
- Check Q_0Q_{-1} = 10 → Perform A = A + (-M):
A = 000101 + 110011 = 111000.
- Perform **ARS**:
A = 111100, Q = 010111, Q_{-1} = 1. SC = 1.

• Cycle 6:
- Check Q_0Q_{-1} = 11 → Perform **ARS only**:
A = 111110, Q = 001011, Q_{-1} = 1. SC = 0. Stop.

Result Verification:

The final combined register contents are A Q = 111110001011_2.

Since the MSB is 1, the result is negative. Let's find its magnitude via 2's complement:

1. Invert all bits → 000001110100
2. Add 1 → 000001110101_2
3. Convert to decimal → $2^6 + 2^5 + 2^4 + 2^2 + 2^0 = 64 + 32 + 16 + 4 + 1 = 117$.

Therefore, the value is -117.

Multiplying (+13) × (-9) yields -117. The trace is mathematically 100% correct!

Q.6 **Perform binary division using restoring and non-restoring division methods with a suitable example.**

Bloom's Taxonomy: 3(Applying) | Course Outcome: CO2

Answer:

Let us perform binary division of **11 divided by 4** (Dividend Q = 11 = `1011`, Divisor M = 4 = `0100`, size n = 4) using both methods.

Method 1: Restoring Division (Step-by-Step):

• Init: A = 0000, Q = 1011, M = 0100, -M = 1100

• Step 1:
1. Shift Left AQ → A = 0001, Q = 011_
2. Subtract M (A = A + (-M)) → A = 0001 + 1100 = 1101.
3. Since A is negative (MSB=1), set Q_0 = 0, and restore A (A = A + M) → A = 1101 + 0100 = 0001.
State: A = 0001, Q = 0110.

• Step 2:
1. Shift Left AQ → A = 0010, Q = 110_
2. Subtract M → A = 0010 + 1100 = 1110.
3. Since A is negative, set Q_0 = 0, and restore A → A = 1110 + 0100 = 0010.
State: A = 0010, Q = 1100.

• Step 3:
1. Shift Left AQ → A = 0101, Q = 100_
2. Subtract M → A = 0101 + 1100 = 0001.
3. Since A is positive (MSB=0), set Q_0 = 1, no restore needed.
State: A = 0001, Q = 1001.

• Step 4:
1. Shift Left AQ → A = 0011, Q = 011_
2. Subtract M → A = 0011 + 1100 = 1111.
3. Since A is negative, set Q_0 = 0, and restore A → A = 1111 + 0100 = 0011.
State: A = 0011, Q = 0010.

• Restoring Result: Quotient Q = 0010_2 (2), Remainder A = 0011_2 (3).

Method 2: Non-Restoring Division (Step-by-Step):

In Non-Restoring Division, we do not perform restoring additions. If A is positive, we shift left and subtract M. If A is negative, we shift left and add M.

• Init: A = 0000, Q = 1011, M = 0100, -M = 1100

• Step 1 (A is positive):
1. Shift Left AQ → A = 0001, Q = 011_
2. Subtract M → A = 0001 + 1100 = 1101.
3. Since A is negative, set Q_0 = 0 (no restore).
State: A = 1101, Q = 0110.

• Step 2 (A is negative):
1. Shift Left AQ → A = 1010, Q = 110_
2. Add M → A = 1010 + 0100 = 1110.
3. Since A is negative, set Q_0 = 0.
State: A = 1110, Q = 1100.

• Step 3 (A is negative):
1. Shift Left AQ → A = 1101, Q = 100_
2. Add M → A = 1101 + 0100 = 0001.
3. Since A is positive, set Q_0 = 1.
State: A = 0001, Q = 1001.

• Step 4 (A is positive):
1. Shift Left AQ → A = 0011, Q = 001_
2. Subtract M → A = 0011 + 1100 = 1111.
3. Since A is negative, set Q_0 = 0.
State: A = 1111, Q = 0010.

• Correction Step: Since A is negative at the end, perform A = A + M to restore final remainder:
A = 1111 + 0100 = 0011.
Final Corrected State: A = 0011, Q = 0010.

• Non-Restoring Result: Quotient Q = 0010_2 (2), Remainder A = 0011_2 (3).

Q.7 Analyze the difference between fixed-point and floating-point number representation in terms of range and precision.

Bloom's Taxonomy: 4(Analyzing) | Course Outcome: CO2

Answer:

Below is a detailed analysis contrasting Fixed-Point and Floating-Point number representations across range and precision boundaries:

• Range of Values:

- Fixed-Point: Has a very narrow, restricted range of values because the radix point is fixed at a static bit position. Example: An 8-bit fixed-point format with a 4-bit integer part can only represent values between -8.0 and +7.93.
- Floating-Point: Offers an exceptionally vast range of values because the exponent field dynamically shifts the radix point. Example: Standard 32-bit single-precision float can represent values from $\pm 1.18 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$.

• Precision (Resolution):

- Fixed-Point: Maintains constant, absolute precision across the entire range of values. The gap between any two consecutive representable numbers is identical.
- Floating-Point: Exhibits variable, relative precision. The absolute precision is extremely high for values close to zero but degrades significantly for extremely large numbers (due to the limited 23-bit mantissa space).

Q.8 **Examine the hardware implementation requirements for arithmetic unit design.**

Bloom's Taxonomy: 4(Analyzing) | Course Outcome: CO2

Answer:

Designing a high-performance **Arithmetic Unit** stage requires satisfying key hardware requirements to support multiple parallel microoperations:

- Full Adder Array: A central array of Full Adders (FAs) forms the core mathematical summation engine.
- Input Conditioning Logic (MUX): Rather than routing register outputs directly to the adder inputs, multiplexers are used to condition the inputs first. For example, the Y input of the adder is driven by a multiplexer that selects between B_i (addition), B_i' (subtraction), 0 (transfer), or 1 (decrement).
- Complementer Arrays: Controlled XOR gates or inverters to generate 1's complements of registers instantly.
- Control Select Lines: Dedicated selection pins (S_1, S_0, C_{in}) driven directly by control memory to dynamically set the active arithmetic mode.
- Bus Latches & Registers: Bidirectional buffers and registers (like AC and DR) with Load lines tied to system clock edges to capture the result safely without race hazards.

Q.9 **Evaluate the efficiency of Booth's algorithm compared to conventional multiplication methods.**

Bloom's Taxonomy: 5(Evaluating) | Course Outcome: CO2

Answer:

Booth's Multiplication Algorithm offers significant efficiency improvements over conventional shift-and-add multiplication methods:

- Shift-and-Add Overhead: Conventional multiplication requires examining every single bit of the multiplier. If a bit is 1, a full binary addition must occur. For an n-bit multiplier, this results in an average of n/2 additions.
- Booth's Grouping Shortcut: Booth's algorithm exploits the fact that a string of consecutive 1s in the multiplier (e.g. `00111100`, which represents 60) can be computed with a single subtraction at the start of the string (`$2^6$`) and a single addition at the end (`$2^2$`), rather than performing 4 separate additions (64 − 4 = 60).

• Efficiency Gains:

- Best-Case Speedup: If the multiplier contains long sequences of 1s or 0s, the number of addition/subtraction cycles drops dramatically, reducing arithmetic hardware latency.
- Unified Signed Handling: Unlike conventional methods (which require separate pre-processing steps to handle negative signs), Booth's algorithm naturally multiplies positive and negative 2's complement numbers without any sign adjustments, streamlining execution pathways.

Q.10 **Design an arithmetic unit capable of performing addition, subtraction, multiplication, and division operations.**

Bloom's Taxonomy: 6(Creating) | Course Outcome: CO2

Answer:

To design a complete, multi-functional arithmetic unit capable of performing **addition, subtraction, multiplication, and division**, we organize the hardware into modular execution pipelines:

Functional Block Design of the Complete Arithmetic Unit:

- Adder-Subtractor Core: An n-bit Parallel Binary Adder combined with XOR gates on the register B inputs. A control line **Sub** decides the function: if Sub = 0, it performs addition (A + B); if Sub = 1, it complements B and sets carry-in C_0 = 1, performing 2's complement subtraction (A + B' + 1).
- Booth's Multiplier Pipeline: Incorporates a dedicated Shift Register (A-Q-Q_{-1}) and an execution sequencer that automatically controls the Adder-Subtractor core based on Q_0Q_{-1} bit transitions, completing signed multiplications in n clock cycles.
- Restoring/Non-Restoring Division Module: Integrates left-shift path lines on registers A and Q, feeding the output of the adder-subtractor back to A based on the sign of the remainder (MSB of A), enabling hardware division.
- Internal Control Bus Matrix: Directs register load enable signals, MUX input routing, and clock gates to orchestrate data flows among all processing modules.